- *Missing time zone and culture data*—Linux distributions don't always come with time zone or culture data, which can cause localization problems and exceptions at runtime. You can install the time zone data using your distribution's package manager.[1] It also may be organized differently. The hierarchy of Norwegian cultures is different in Linux, for example.
- *Different directory structures*—Linux distributions use a different folder structure from Windows, so you need to bear that in mind if your app hardcodes paths. In particular, consider differences in temporary/cache folders.

The preceding list is not exhaustive by any means, but as long as you set up `ForwardedHeadersMiddleware` and take care to use cross-platform constructs like `Path.Combine`, you shouldn't have too many problems running your applications on Linux. But configuring a reverse proxy isn't the simplest of activities, so wherever you're planning on hosting your app, I suggest checking the documentation for guidance at http://mng.bz/1qM1.

27.4 Configuring the URLs for your application

At this point, you've deployed an application, but there's one aspect you haven't configured: the URLs for your application. When you're using IIS as a reverse proxy, you don't have to worry about this inside your app. IIS integration with the ASP.NET Core Module works by dynamically creating a URL that's used to forward requests between IIS and your app. The hostname you configure in IIS (in figure 27.6) is the URL that external users